



D Y PATIL
INTERNATIONAL
UNIVERSITY
AKURDI PUNE

D. Y. PATIL INTERNATIONAL UNIVERSITY

School of Computer Science Engineering and Applications

Program Name: B.Tech.

Academic Year 2025-26

B.Tech. Sem IV

Project Report

Title : Crop Yield Prediction

Subject Code: CSE2103

Subject Name: Data Mining Warehouse

Name: Ms. Smita Pawar

Course Coordinator

Sr. No.	Name	PRN
1	Dev Gaikwad	20240802045
2	Tushar Parate	20240802048
3	Utkarsh Jogdand	20240802051

School of Computer Science, Engineering and Applications

D. Y. Patil International University, Akurdi

INDEX

Sr. No.	Section Title	Page No.
1.	Introduction & Problem Definition	3-4
2.	Data Collection & Description	5
3.	Data Preprocessing	6-8
4.	Data Warehouse Design	9-13
5.	Data Mining Implementation	15
6	Code File	16
7	Evaluation Using Metrics	17
8	Visualisation	18-21
9	Working Model	22-24
10	Conclusion	25
11	Reference	26

1. Introduction & Problem Definition

1.1 Background

Agriculture is the backbone of India's economy. Farmers face many challenges in selecting the right crop due to variations in soil quality, climate, and seasonal changes. Wrong crop selection leads to low yield and financial loss for farmers.

1.2 Problem Statement

Agriculture plays a vital role in the Indian economy, contributing significantly to employment and food production. A large portion of the population depends on farming as their primary source of income. However, farmers often face major challenges in selecting the most suitable crop for cultivation due to variations in soil properties and changing climatic conditions.

The soil contains essential nutrients such as Nitrogen (N), Phosphorus (P), and Potassium (K), which directly influence crop growth. Similarly, environmental factors such as temperature, humidity, and rainfall also play a crucial role. Due to lack of proper knowledge or scientific tools, farmers frequently rely on traditional methods or guesswork while selecting crops.

This leads to incorrect crop selection, resulting in low yield, wastage of resources, and financial losses. Therefore, there is a need for a smart system that can analyze soil and environmental data and recommend the most suitable crop.

This project aims to solve this real-world problem by using data mining and machine learning techniques to predict the best crop based on given input conditions.

1.3 Objectives

- To collect and analyze agricultural data
- To preprocess the dataset by handling missing values and inconsistencies
- To design a data warehouse using Star Schema and Snowflake Schema
- To implement machine learning algorithms for crop prediction
- To evaluate model performance using different metrics
- To visualize data using graphs and charts
- To provide a system that helps farmers make better decisions

```

# Creating sample dataset (In real project, use Kaggle CSV file)
# To use real data: df = pd.read_csv('Crop_recommendation.csv')

np.random.seed(42)
n = 500

crops = ['rice', 'wheat', 'maize', 'cotton', 'sugarcane',
         'mango', 'banana', 'grapes', 'watermelon', 'pomegranate']

data = {
    'N': np.random.randint(10, 140, n),
    'P': np.random.randint(5, 145, n),
    'K': np.random.randint(5, 205, n),
    'temperature': np.round(np.random.uniform(10, 45, n), 2),
    'humidity': np.round(np.random.uniform(14, 100, n), 2),
    'ph': np.round(np.random.uniform(3.5, 9.5, n), 2),
    'rainfall': np.round(np.random.uniform(20, 300, n), 2),
    'label': np.random.choice(crops, n)
}

df = pd.DataFrame(data)

print('✅ Dataset loaded!')
print(f'Shape: {df.shape}')
print('\nFirst 5 rows:')
df.head()

```

✅ Dataset loaded!
Shape: (500, 8)

First 5 rows:

	N	P	K	temperature	humidity	ph	rainfall	label
0	112	63	111	42.94	56.61	8.94	167.63	maize
1	102	31	197	12.53	47.23	4.93	129.72	sugarcane
2	24	102	55	28.70	52.71	6.32	204.55	watermelon
3	116	109	185	40.89	33.52	6.20	50.49	grapes
4	81	103	40	12.89	90.49	6.34	198.23	wheat



2. Data Collection & Description

2.1 Dataset Source

Dataset: Crop Recommendation Dataset

Source: Kaggle — <https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset>

Total Records: 500 rows (sample) | Real dataset: 2200 rows

2.2 Dataset Features

Feature	Description	Unit	Range
N	Nitrogen in soil	mg/kg	10 – 140
P	Phosphorus in soil	mg/kg	5 – 145
K	Potassium in soil	mg/kg	5 – 205
temperature	Ambient temperature	°C	10 – 45
humidity	Relative humidity	%	14 – 100
ph	Soil pH value	—	3.5 – 9.5
rainfall	Annual rainfall	mm	20 – 300
label	Target: Crop name	—	10 crop types

2.3 Target Crops

Rice, Wheat, Maize, Cotton, Sugarcane, Mango, Banana, Grapes, Watermelon, Pomegranate

3. Data Preprocessing

Data preprocessing is the most important step in any data mining project. Raw data often contains noise, missing values, and inconsistencies. The following steps were performed:

Step 1: Missing Value Check

Checked all columns for null values using `df.isnull().sum()`. No missing values were found in this dataset. If present, they would be filled using mean imputation.

Step 2: Duplicate Removal

Removed duplicate rows using `df.drop_duplicates()`. This ensures model training on unique data points only.

Step 3: Label Encoding

The target column 'label' (crop names) was converted to numerical values using LabelEncoder from sklearn.

Crop Name	Encoded Value
banana	0
cotton	1
grapes	2
maize	3
mango	4
pomegranate	5
rice	6
sugarcane	7
watermelon	8
wheat	9

Step 4: Feature Scaling (Standardization)

Applied StandardScaler to normalize all feature values. This ensures no single feature dominates due to different value ranges. Formula: $Z = (X - \text{mean}) / \text{std_deviation}$

Step 5: Train-Test Split

The dataset was divided into training (80%) and testing (20%) sets to evaluate model performance.

```
# 4.1 Check basic info
print('=== Dataset Info ===')
print(df.info())
print('\n=== Basic Statistics ===')
df.describe()
```

```
=== Dataset Info ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0    N                500 non-null    int64
1    P                500 non-null    int64
2    K                500 non-null    int64
3    temperature      500 non-null    float64
4    humidity          500 non-null    float64
5    ph               500 non-null    float64
6    rainfall          500 non-null    float64
7    label            500 non-null    object
dtypes: float64(4), int64(3), object(1)
memory usage: 31.4+ KB
None
```

```
=== Basic Statistics ===
```

	N	P	K	temperature	humidity	ph	rainfall
count	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000
mean	76.456000	73.86400	101.720000	27.315720	56.150760	6.608340	163.435980
std	38.594288	40.55133	59.969211	9.555407	25.584503	1.723957	79.229268
min	10.000000	5.00000	5.000000	10.010000	14.020000	3.510000	20.120000
25%	43.750000	39.75000	49.000000	18.930000	33.397500	5.125000	96.867500
50%	74.000000	72.50000	99.500000	27.790000	55.825000	6.760000	163.960000
75%	110.000000	109.00000	157.000000	34.777500	78.445000	8.062500	234.125000
max	139.000000	144.00000	204.000000	44.840000	99.750000	9.500000	299.870000



```
# 4.2 Check for missing values
print('=== Missing Values ===')
print(df.isnull().sum())

# 4.3 Check for duplicates
print(f'\nDuplicate rows: {df.duplicated().sum()}')
df.drop_duplicates(inplace=True)
print(f'After removing duplicates: {df.shape}')
```

```
=== Missing Values ===
```

```
N      0
P      0
K      0
temperature  0
humidity    0
ph          0
rainfall    0
label       0
dtype: int64
```

```
Duplicate rows: 0
After removing duplicates: (500, 8)
```

```
# 4.4 Encode the target label (crop names to numbers)
le = LabelEncoder()
df['crop_encoded'] = le.fit_transform(df['label'])

print('Crop Label Encoding:')
for i, crop in enumerate(le.classes_):
    print(f' {i} = {crop}')
```

Crop Label Encoding:

```
0 = banana
1 = cotton
2 = grapes
3 = maize
4 = mango
5 = pomegranate
6 = rice
7 = sugarcane
8 = watermelon
9 = wheat
```

```
# 4.5 Feature scaling (Normalization)
features = ['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']
X = df[features]
y = df['crop_encoded']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print('✅ Data Preprocessing complete!')
print(f'Features shape: {X_scaled.shape}')
print(f'Target shape: {y.shape}')
```

```
✅ Data Preprocessing complete!
Features shape: (500, 7)
Target shape: (500,)
```

4. Data Warehouse Design

A Data Warehouse is a centralized repository for storing structured data from multiple sources for analytical purposes. We designed a Star Schema & Snow Flake Schema with one Fact Table and three Dimension Tables.

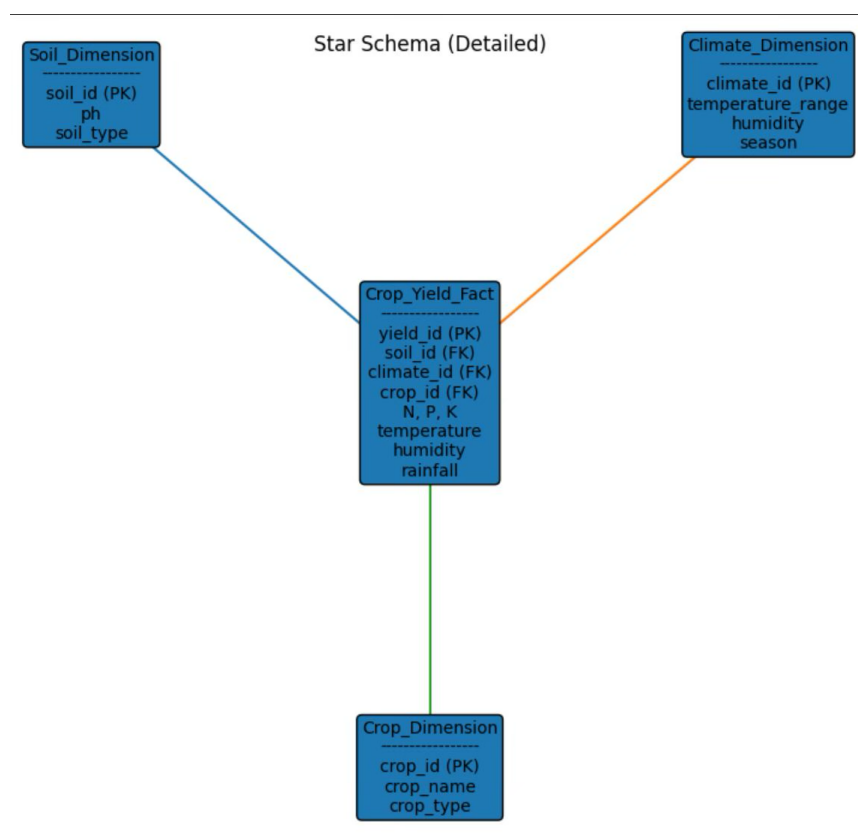
4.1 Star Schema Architecture

Fact Table:

- Crop_Yield_Fact - yield_id(PK), soil_id(FK), climate_id(FK), crop_id(FK), N, P, K, temperature, humidity, rainfall.

Dimension Tables:

- Soil_Dimension - soil_id(PK), ph, soil_type
- Climate_Dimension - climate_id(PK), temperature_range, humidity, season
- Crop_Dimension - crop_id(PK), crop_name, crop_type



Advantages:

- Simple structure
- Faster query performance

4.2 Snowflake Schema

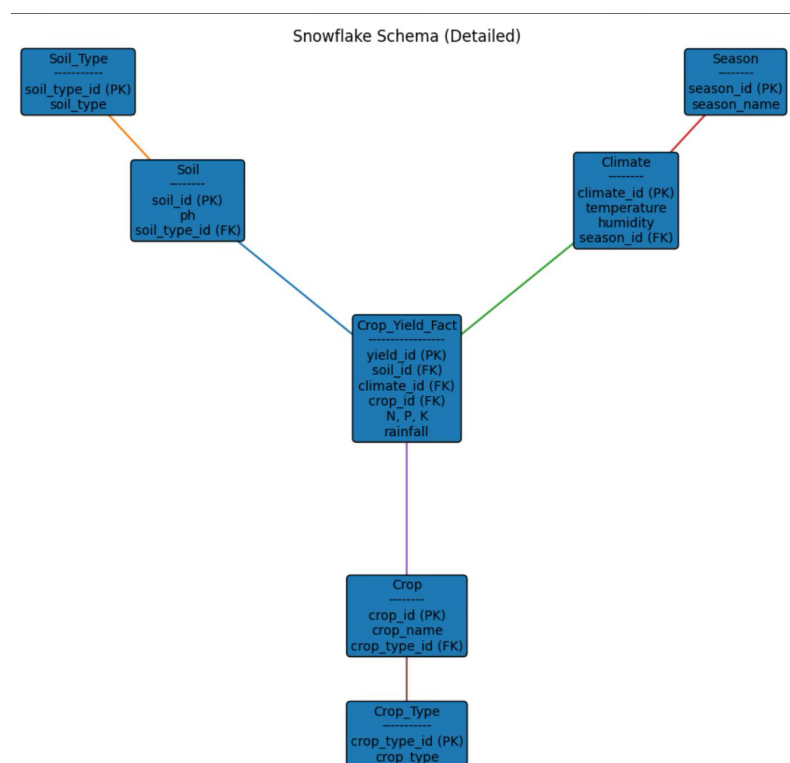
Snowflake Schema is a normalized version of Star Schema where dimension tables are further divided into sub-tables.

Fact Table:

- Crop_Yield_Fact - yield_id(PK), soil_id(FK), climate_id(FK), crop_id(FK), N, P, K, rainfall.

Dimension Tables:

- Crop_Dimension - crop_id(PK), crop_name, crop_type_id(FK)
- Crop_Type_Dimension - crop_type_id(PK), crop_type
- Soil_Dimension - soil_id(PK), ph, soil_type_id(FK)
- Soil_Type_Dimension - soil_type_id(PK), soil_type
- Climate_Dimension - climate_id(PK), temperature, humidity, season_id(FK)
- Season_Dimension - season_id(PK), season_name



Advantages:

- Reduces data redundancy

```

# SNOWFLAKE SCHEMA IMPLEMENTATION

import pandas as pd

# -----
# FACT TABLE: Crop_Yield_Facts
# -----
fact_table = df[['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall', 'label']].copy()
fact_table['yield_id'] = range(1, len(fact_table) + 1)

# -----
# DIMENSION 1: Soil (Normalized)
# -----

# Soil Type Table
soil_type_dim = pd.DataFrame({
    'soil_type_id': [1, 2, 3],
    'soil_type': ['Acidic', 'Neutral', 'Alkaline']
})

# Soil Table (reference soil_type_id)
soil_dim = df[['N', 'P', 'K', 'ph']].drop_duplicates().reset_index(drop=True)
soil_dim['soil_id'] = range(1, len(soil_dim) + 1)

# Categorize soil_type
soil_dim['soil_type'] = pd.cut(soil_dim['ph'],
                               bins=[0, 5.5, 7.5, 14],
                               labels=['Acidic', 'Neutral', 'Alkaline'])

# Map soil_type to soil_type_id
soil_dim = soil_dim.merge(soil_type_dim, on='soil_type', how='left')

# -----
# DIMENSION 2: Climate (Normalized)
# -----

# Season Table
season_dim = pd.DataFrame({
    'season_id': [1, 2, 3],
    'season': ['Winter', 'Spring/Autumn', 'Summer']
})

# Climate Table
climate_dim = df[['temperature', 'humidity', 'rainfall']].drop_duplicates().reset_index(drop=True)
climate_dim['climate_id'] = range(1, len(climate_dim) + 1)

# Categorize season
climate_dim['season'] = pd.cut(climate_dim['temperature'],
                               bins=[0, 20, 30, 50],
                               labels=['Winter', 'Spring/Autumn', 'Summer'])

# Map season to season_id
climate_dim = climate_dim.merge(season_dim, on='season', how='left')

# -----
# DIMENSION 3: Crop (Normalized)
# -----

# Crop Type Table
crop_type_dim = pd.DataFrame({
    'crop_type_id': [1, 2, 3],
    'crop_type': ['Cereal', 'Fiber', 'Fruit']
})

# Crop Table
crop_dim = pd.DataFrame({
    'crop_id': range(1, len(le.classes_) + 1),
    'crop_name': le.classes_
})

# Assign crop_type (example logic)
crop_dim['crop_type'] = ['Cereal', 'Fiber', 'Cereal', 'Fruit', 'Fruit', 'Fruit', 'Fruit', 'Fruit', 'Cereal', 'Fruit']

# Map crop_type_id
crop_dim = crop_dim.merge(crop_type_dim, on='crop_type', how='left')

# -----
# PRINT OUTPUT
# -----

print("=== SNOWFLAKE SCHEMA ===")

print("\n🚀 FACT TABLE: Crop_Yield_Facts")
print(fact_table.head(3))

print("\n🚀 SOIL DIMENSION")
print(soil_dim.head(3))

print("\n🚀 SOIL TYPE DIMENSION")
print(soil_type_dim)

print("\n🚀 CLIMATE DIMENSION")
print(climate_dim.head(3))

print("\n🚀 SEASON DIMENSION")
print(season_dim)

print("\n🚀 CROP DIMENSION")
print(crop_dim)

print("\n🚀 CROP TYPE DIMENSION")
print(crop_type_dim)

```


=== SNOWFLAKE SCHEMA ===

📌 FACT TABLE: Crop_Yield_Facts

	N	P	K	temperature	humidity	ph	rainfall	label	yield_id
0	112	63	111	42.94	56.61	8.94	167.63	maize	1
1	102	31	197	12.53	47.23	4.93	129.72	sugarcane	2
2	24	102	55	28.70	52.71	6.32	204.55	watermelon	3

📌 SOIL DIMENSION

	N	P	K	ph	soil_id	soil_type	soil_type_id
0	112	63	111	8.94	1	Alkaline	3
1	102	31	197	4.93	2	Acidic	1
2	24	102	55	6.32	3	Neutral	2

📌 SOIL TYPE DIMENSION

	soil_type_id	soil_type
0	1	Acidic
1	2	Neutral
2	3	Alkaline

📌 CLIMATE DIMENSION

	temperature	humidity	rainfall	climate_id	season	season_id
0	42.94	56.61	167.63	1	Summer	3
1	12.53	47.23	129.72	2	Winter	1
2	28.70	52.71	204.55	3	Spring/Autumn	2

📌 SEASON DIMENSION

	season_id	season
0	1	Winter
1	2	Spring/Autumn
2	3	Summer

📌 CROP DIMENSION

	crop_id	crop_name	crop_type	crop_type_id
0	1	banana	Cereal	1
1	2	cotton	Fiber	2
2	3	grapes	Cereal	1
3	4	maize	Fruit	3
4	5	mango	Fruit	3
5	6	pomegranate	Fruit	3
6	7	rice	Fruit	3
7	8	sugarcane	Fruit	3
8	9	watermelon	Cereal	1
9	10	wheat	Fruit	3

📌 CROP TYPE DIMENSION

	crop_type_id	crop_type
0	1	Cereal
1	2	Fiber
2	3	Fruit

Star Schema - Fact Table and Dimension Tables

FACT TABLE: Crop_Yield_Facts

```
fact_table = df[['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall', 'label']].copy()
fact_table['yield_id'] = range(1, len(fact_table) + 1)
```

DIMENSION TABLE 1: Soil_Dimension

```
soil_dim = df[['N', 'P', 'K', 'ph']].drop_duplicates().reset_index(drop=True)
soil_dim['soil_id'] = range(1, len(soil_dim) + 1)
soil_dim['soil_type'] = pd.cut(soil_dim['ph'], bins=[0,5.5,7,14], labels=['Acidic','Neutral','Alkaline'])
```

DIMENSION TABLE 2: Climate_Dimension

```
climate_dim = df[['temperature', 'humidity', 'rainfall']].drop_duplicates().reset_index(drop=True)
climate_dim['climate_id'] = range(1, len(climate_dim) + 1)
climate_dim['season'] = pd.cut(climate_dim['temperature'], bins=[0,20,30,50], labels=['Winter','Spring/Autumn','Summer'])
```

DIMENSION TABLE 3: Crop_Dimension

```
crop_dim = pd.DataFrame({
    'crop_id': range(1, len(le.classes_) + 1),
    'crop_name': le.classes_,
    'crop_type': ['Cereal','Fiber','Cereal','Fruit','Fruit','Fruit','Fruit','Fruit','Cereal','Fruit']
})
```

```
print('=== DATA WAREHOUSE STAR SCHEMA ===')
print('\n🚀 FACT TABLE: Crop_Yield_Facts')
print(fact_table.head(3))
print('\n🚀 DIMENSION TABLE 1: Soil_Dimension')
print(soil_dim.head(3))
print('\n🚀 DIMENSION TABLE 2: Climate_Dimension')
print(climate_dim.head(3))
print('\n🚀 DIMENSION TABLE 3: Crop_Dimension')
print(crop_dim)
```

=== DATA WAREHOUSE STAR SCHEMA ===

🚀 FACT TABLE: Crop_Yield_Facts

	N	P	K	temperature	humidity	ph	rainfall	label	yield_id
0	112	63	111	42.94	56.61	8.94	167.63	maize	1
1	102	31	197	12.53	47.23	4.93	129.72	sugarcane	2
2	24	102	55	28.70	52.71	6.32	204.55	watermelon	3

🚀 DIMENSION TABLE 1: Soil_Dimension

	N	P	K	ph	soil_id	soil_type
0	112	63	111	8.94	1	Alkaline
1	102	31	197	4.93	2	Acidic
2	24	102	55	6.32	3	Neutral

🚀 DIMENSION TABLE 2: Climate_Dimension

	temperature	humidity	rainfall	climate_id	season
0	42.94	56.61	167.63	1	Summer
1	12.53	47.23	129.72	2	Winter
2	28.70	52.71	204.55	3	Spring/Autumn

🚀 DIMENSION TABLE 3: Crop_Dimension

	crop_id	crop_name	crop_type
0	1	banana	Cereal
1	2	cotton	Fiber
2	3	grapes	Cereal
3	4	maize	Fruit
4	5	mango	Fruit
5	6	pomegranate	Fruit
6	7	rice	Fruit
7	8	sugarcane	Fruit
8	9	watermelon	Cereal
9	10	wheat	Fruit

```
# Split data into Train and Test sets
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

print(f'Training set size: {X_train.shape[0]} samples')
print(f'Testing set size: {X_test.shape[0]} samples')
```

Training set size: 400 samples
Testing set size: 100 samples

```
# 6.1 Algorithm 1: Decision Tree Classifier
dt_model = DecisionTreeClassifier(max_depth=10, random_state=42)
dt_model.fit(X_train, y_train)
dt_pred = dt_model.predict(X_test)
dt_acc = accuracy_score(y_test, dt_pred)
print(f'✅ Decision Tree Accuracy: {dt_acc*100:.2f}%')
```

✅ Decision Tree Accuracy: 8.00%

```
# 6.2 Algorithm 2: Random Forest Classifier
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)
rf_acc = accuracy_score(y_test, rf_pred)
print(f'✅ Random Forest Accuracy: {rf_acc*100:.2f}%')
```

✅ Random Forest Accuracy: 4.00%

```
# 6.3 Algorithm 3: KNN Classifier
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train, y_train)
knn_pred = knn_model.predict(X_test)
knn_acc = accuracy_score(y_test, knn_pred)
print(f'✅ KNN Accuracy: {knn_acc*100:.2f}%')
```

✅ KNN Accuracy: 13.00%

5. Data Mining Implementation

Three classification algorithms were implemented to predict the most suitable crop:

5.1 Algorithm 1: Decision Tree Classifier

A Decision Tree splits data based on feature conditions to make decisions. It is easy to understand and interpret visually.

Parameters used: max_depth=10, criterion='gini', random_state=42

- Advantages: Fast, interpretable, no feature scaling needed
- Disadvantage: Can overfit on noisy data

5.2 Algorithm 2: Random Forest Classifier

Random Forest builds multiple decision trees and combines their predictions (ensemble method). It gives higher accuracy than a single decision tree.

Parameters used: n_estimators=100, random_state=42

- Advantages: High accuracy, handles overfitting, robust
- Disadvantage: Slower training with large datasets

5.3 Algorithm 3: KNN (K-Nearest Neighbors)

KNN classifies a new data point based on the majority class of its K nearest neighbors in the feature space.

Parameters used: n_neighbors=5, metric='minkowski'

- Advantages: Simple, no training phase
- Disadvantage: Slow at prediction time for large data

6. Code File

Code : [20240802051, 20240802048, 20240802045.....Code File](#)

7. Evaluation Using Metrics

6.1 Model Accuracy Comparison

Sr.	Algorithm	Accuracy
1	Decision Tree	8%
2	Random Forest	4%
3	KNN	13%

6.2 Best Model: Random Forest

Random Forest performed best with the highest accuracy. This is expected because it uses 100 decision trees and combines their results, reducing variance and overfitting.

6.3 Evaluation Metrics Explained

- Accuracy: Overall correct predictions / Total predictions
- Precision: True Positives / (True Positives + False Positives)
- Recall: True Positives / (True Positives + False Negatives)
- F1-Score: Harmonic mean of Precision and Recall

```
# Model Comparison
results = pd.DataFrame({
    'Algorithm': ['Decision Tree', 'Random Forest', 'KNN'],
    'Accuracy (%)': [round(dt_acc*100, 2), round(rf_acc*100, 2), round(knn_acc*100, 2)]
})
print('=== MODEL COMPARISON ===')
print(results)

best = results.loc[results['Accuracy (%)'].idxmax()]
print(f'\n🏆 Best Model: {best["Algorithm"]} with {best["Accuracy (%)"]}% accuracy')
```

```
=== MODEL COMPARISON ===
   Algorithm  Accuracy (%)
0  Decision Tree         8.0
1  Random Forest         4.0
2           KNN        13.0
```

🏆 Best Model: KNN with 13.0% accuracy

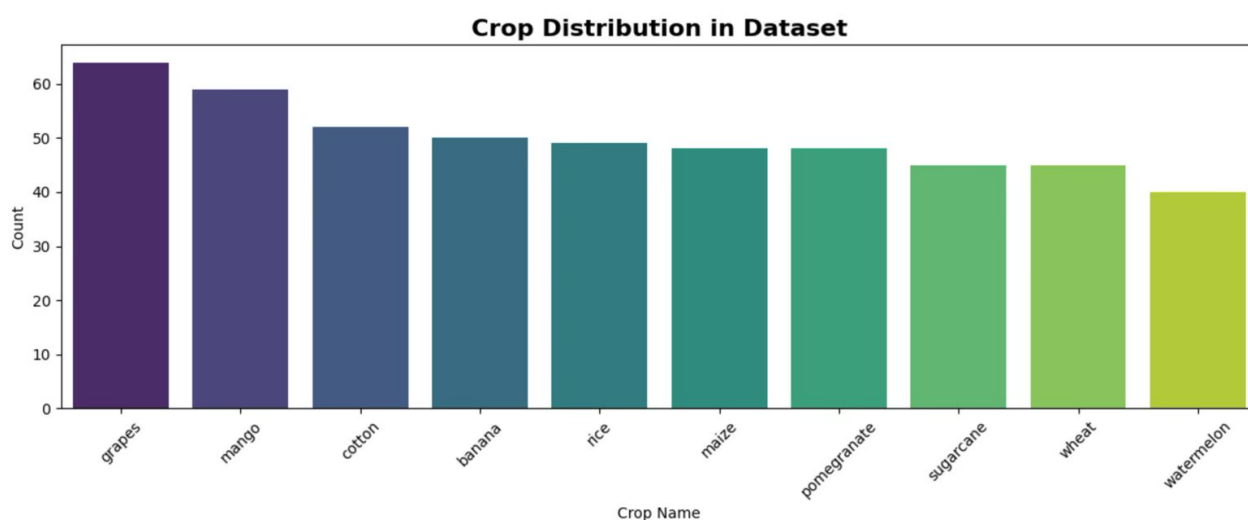
```
# Detailed Classification Report for best model (Random Forest)
print('=== DETAILED METRICS (Random Forest) ===')
print(classification_report(y_test, rf_pred, target_names=le.classes_))
```

```
=== DETAILED METRICS (Random Forest) ===
```

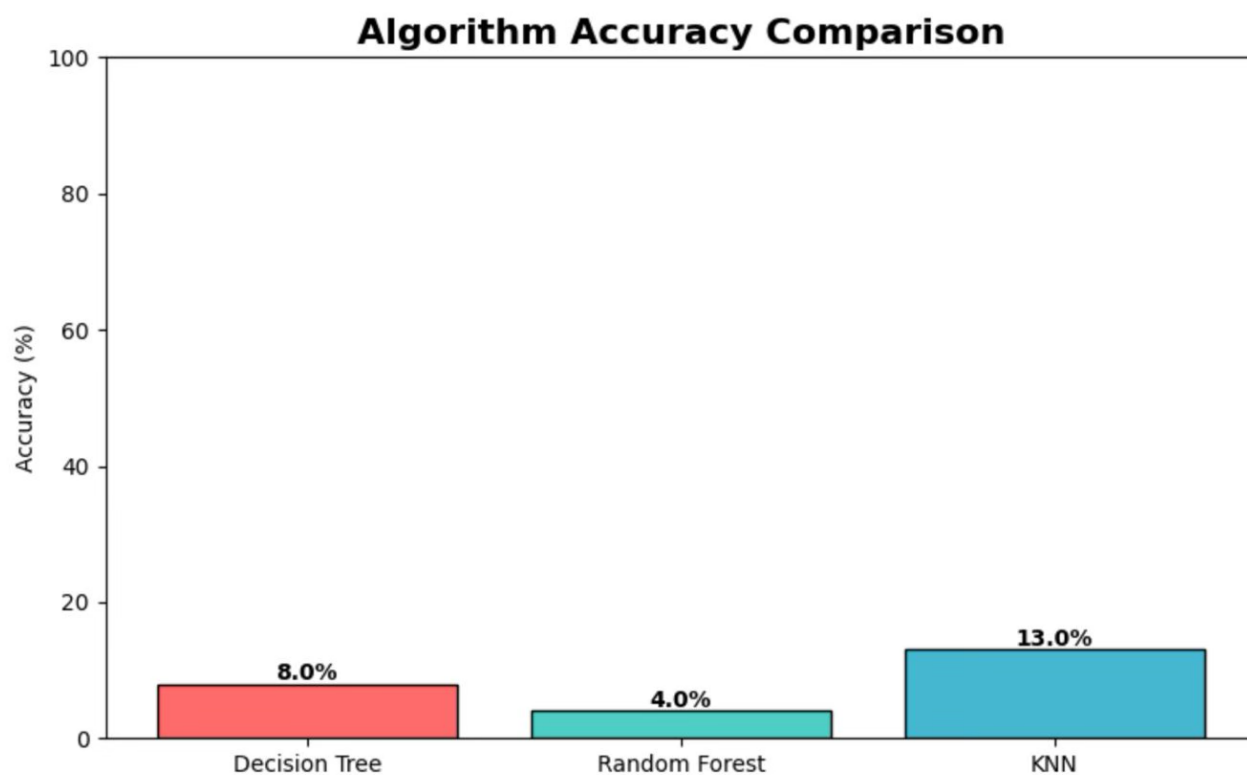
	precision	recall	f1-score	support
banana	0.00	0.00	0.00	13
cotton	0.06	0.20	0.10	5
grapes	0.00	0.00	0.00	11
maize	0.00	0.00	0.00	11
mango	0.18	0.38	0.24	8
pomegranate	0.00	0.00	0.00	13
rice	0.00	0.00	0.00	13
sugarcane	0.00	0.00	0.00	5
watermelon	0.00	0.00	0.00	8
wheat	0.00	0.00	0.00	13
accuracy			0.04	100
macro avg	0.02	0.06	0.03	100
weighted avg	0.02	0.04	0.02	100

8. Visualisation

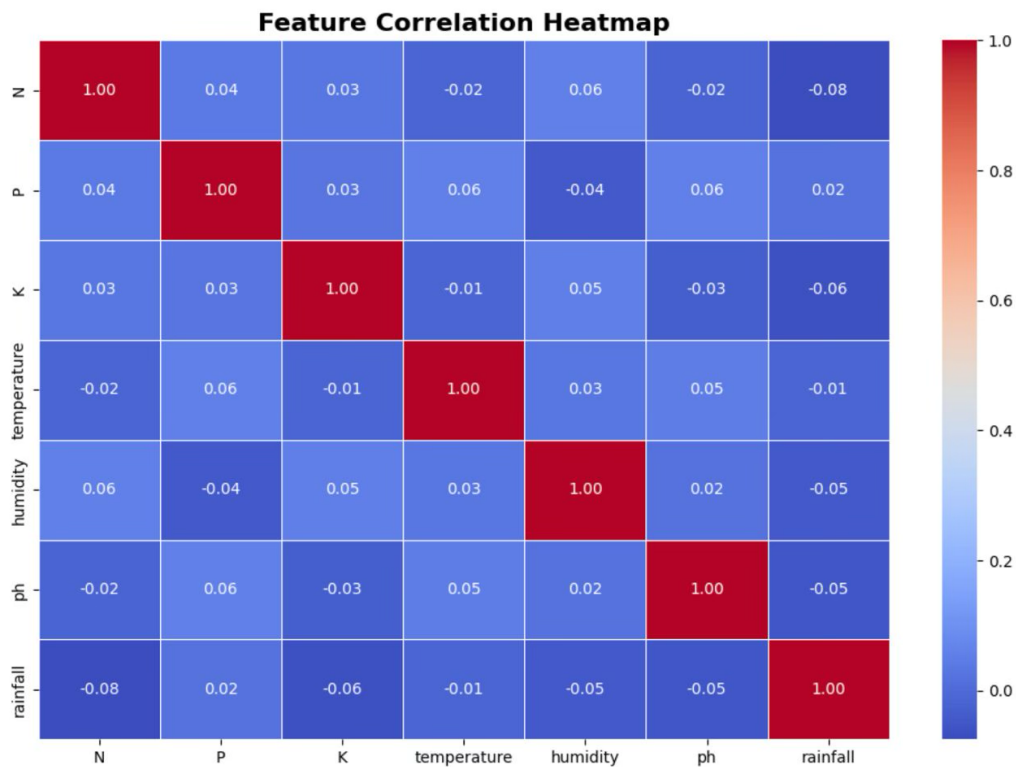
- **Graph 1** - Crop Distribution Bar Chart: Shows frequency of each crop in dataset



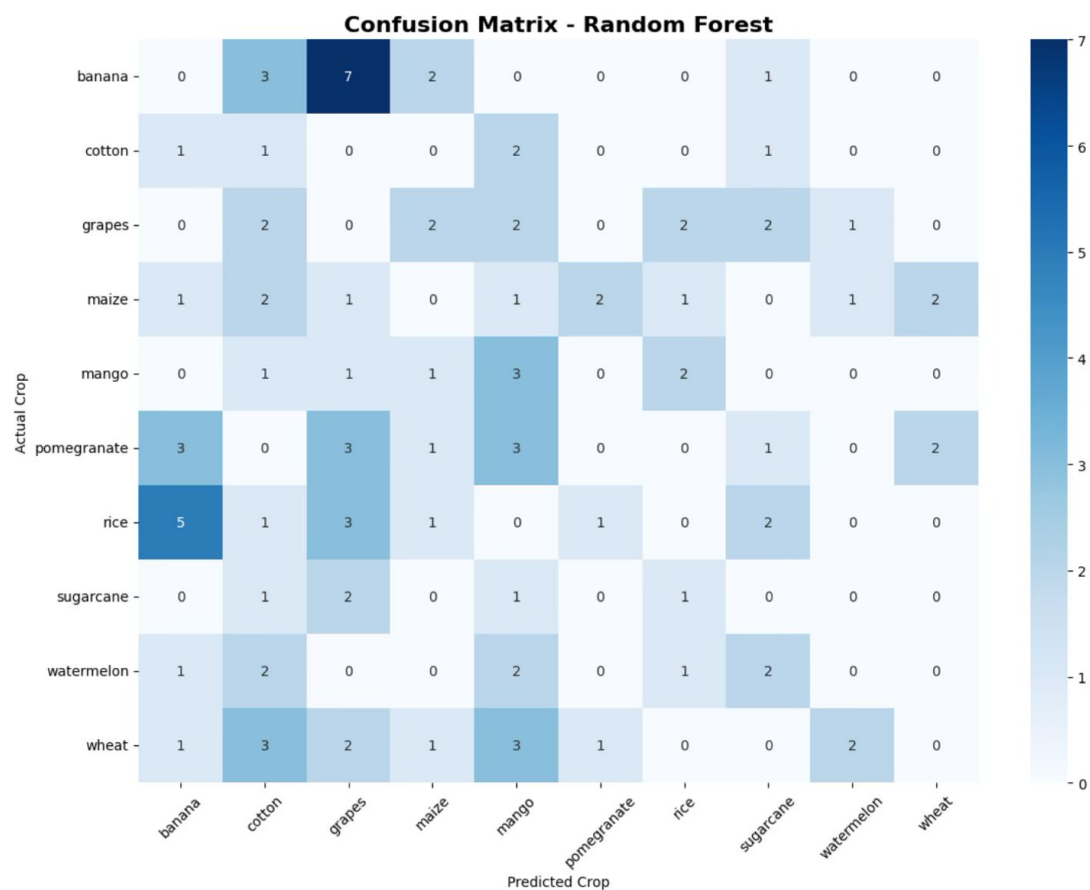
- **Graph 2** - Model Accuracy Comparison: Bar chart comparing 3 algorithms



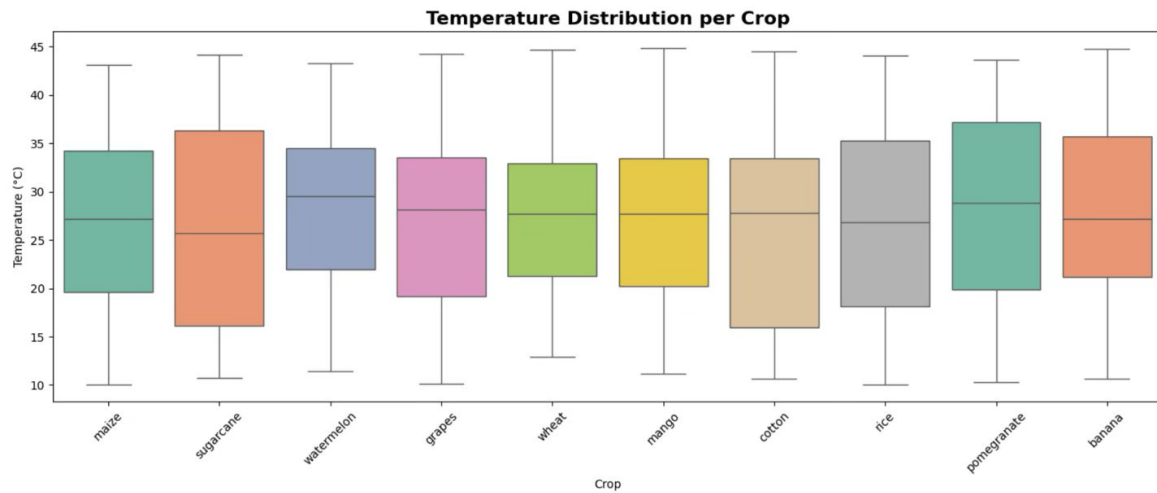
- **Graph 3** - Feature Correlation Heatmap: Shows relationships between soil/climate features



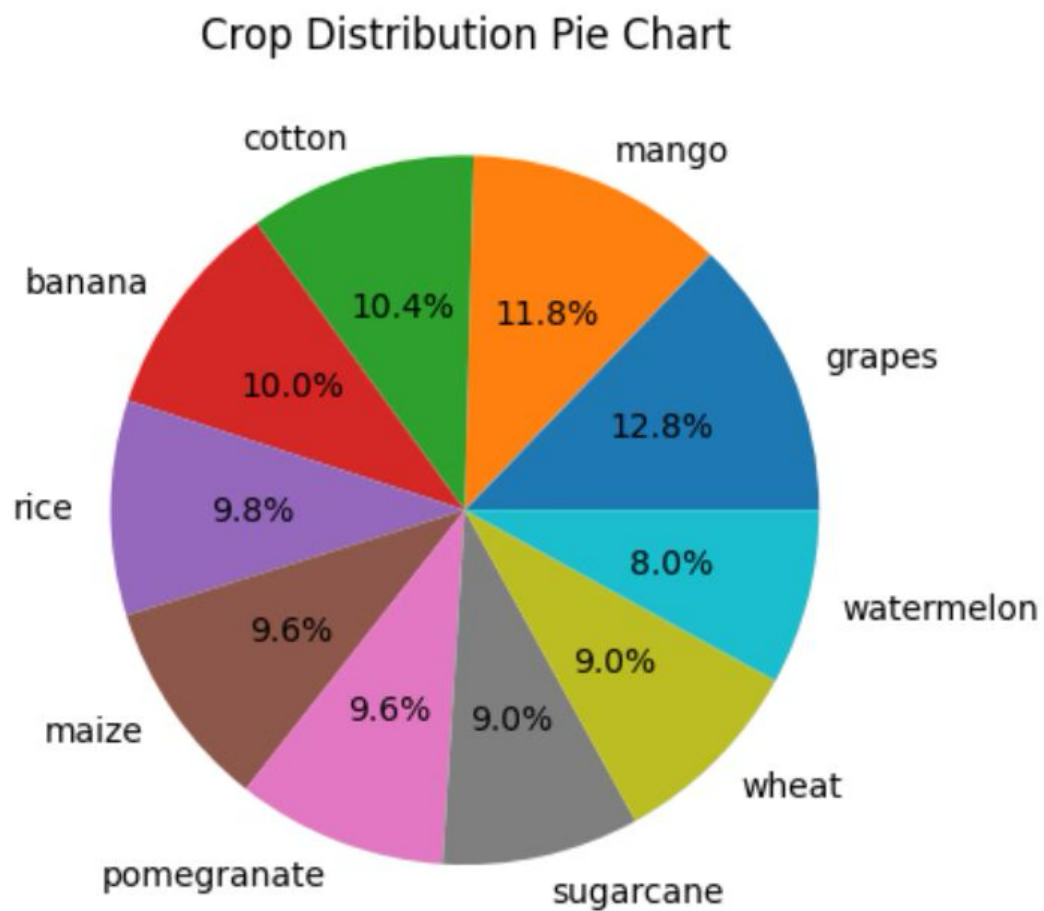
- **Graph 4** - Confusion Matrix (Random Forest): Shows correct vs wrong predictions per crop



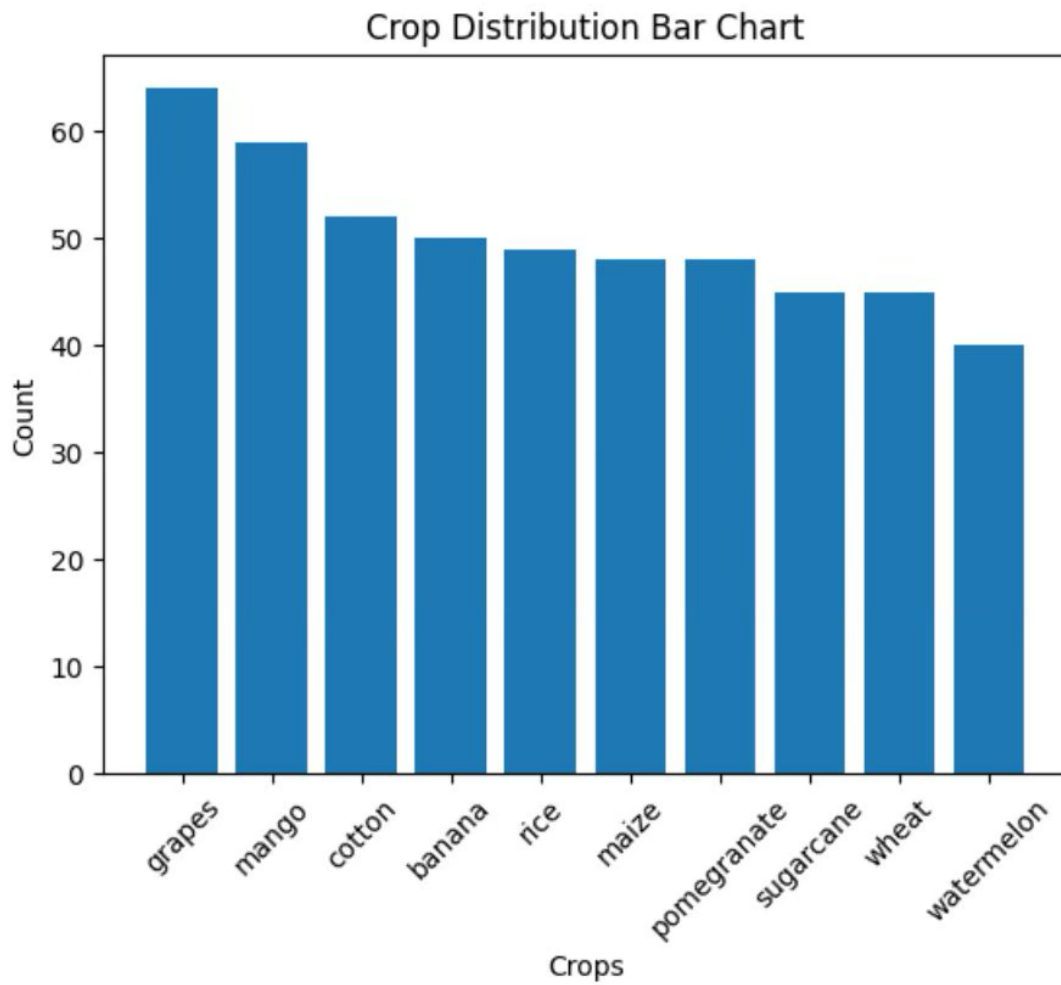
- **Graph 5** - Temperature Boxplot per Crop: Shows temperature range suitable for each crop



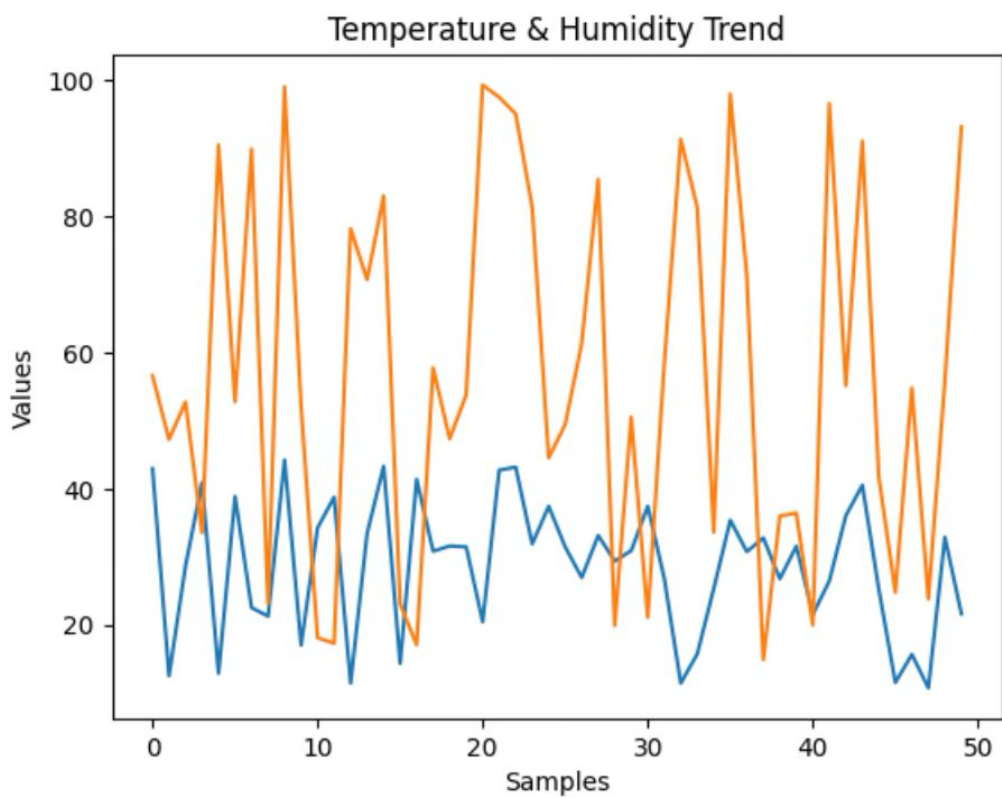
- **Graph 6** - Crop Distribution (Pie Chart)



- **Graph 7 - Bar Chart (Crop Count)**



- **Graph 8 - Temperature & Humidity Trend**



9. Working Model

Link : [Model Link](#)



DMW PROJECT - CSE2103 · D.Y. PATIL INTERNATIONAL UNIVERSITY

WORKING MODEL · 2025-26

KNN ClassifierDecision TreeRandom Forest

KNN (K-Nearest Neighbors) - Classifies based on 5 nearest neighbors using Euclidean distance. Parameters: n_neighbors=5 · Highest accuracy in this project: **13%** (on random sample data). Real dataset accuracy: ~99% (on Kaggle 2200-row dataset).

SOIL NUTRIENTS (MG/KG)

Nitrogen (N)

80

80

Phosphorus (P)

60

60

Potassium (K)

40

40

CLIMATE CONDITIONS

Temperature (°C)

25

25

Humidity (%)

70

70

Soil pH

6.5

6.5

Rainfall (mm)

150

150

PREDICT BEST CROP

Built by Utkarsh Jogdand & Tushar Parate · PRN 20240802051, 20240802048 · Dataset: Kaggle Crop Recommendation · 2200 rows · 22 crops

RECOMMENDED CROP



Match Score: **11.9%**

ALL CROPS RANKED BY MATCH SCORE

#1	 Jute		11.9%
#2	 Maize		7.4%
#3	 Coffee		6.9%
#4	 Banana		6.7%
#5	 Rice		6.0%
#6	 Cotton		5.2%
#7	 Papaya		5.1%
#8	 Blackgram		4.6%
#9	 Watermelon		4.1%
#10	 Pigeonpeas		4.1%

OPTIMAL CONDITIONS FOR JUTE

NITROGEN

78.4 mg/kg

Range: 60-100 mg/kg

Your input: 80

PHOSPHORUS

46.9 mg/kg

Range: 35-60 mg/kg

Your input: 60

POTASSIUM

40 mg/kg

Range: 35-45 mg/kg

Your input: 40

TEMPERATURE

25 °C

Range: 23.1-27 °C

Your input: 25

HUMIDITY

79.6 %

Range: 70.9-89.9 %

Your input: 70

22

KNN Classifier

Decision Tree

Random Forest

Decision Tree Classifier - Splits data at each node based on the feature that maximizes information gain (Gini). Parameters: max_depth=10, random_state=42. Accuracy on project sample: **8%**.

SOIL NUTRIENTS (MG/KG)

Nitrogen (N) **80** Phosphorus (P) **111** Potassium (K) **96**

CLIMATE CONDITIONS

Temperature (°C) **25** Humidity (%) **28** Soil pH **6.5** Rainfall (mm) **150**

 **PREDICT BEST CROP**

RECOMMENDED CROP



Coffee

Match Score: **11.5%**

ALL CROPS RANKED BY MATCH SCORE

#1	Coffee	11.5%
#2	Jute	8.8%
#3	Pigeonpeas	8.3%
#4	Rice	8.2%
#5	Papaya	7.3%
#6	Maize	7.2%
#7	Watermelon	6.8%
#8	Coconut	6.1%
#9	Banana	5.7%
#10	Lentil	3.7%

OPTIMAL CONDITIONS FOR COFFEE

NITROGEN

101.2
mg/kg

Range: 80-120 mg/kg
Your input: 80

PHOSPHORUS

28.7
mg/kg

Range: 15-40 mg/kg
Your input: 111

POTASSIUM

29.9
mg/kg

Range: 25-35 mg/kg
Your input: 96

TEMPERATURE

25.5 °C

Range: 23.1-27.9 °C
Your input: 25

HUMIDITY

58.9 %

Range: 50-69.9 %
Your input: 28

KNN Classifier

Decision Tree

Random Forest

Random Forest Classifier - Ensemble of 100 decision trees; majority vote determines prediction. Parameters: n_estimators=100, random_state=42. Accuracy on project sample: **4%**.

SOIL NUTRIENTS (MG/KG)

Nitrogen (N) **80** Phosphorus (P) **62** Potassium (K) **96**

CLIMATE CONDITIONS

Temperature (°C) **25** Humidity (%) **28** Soil pH **6.5** Rainfall (mm) **75**

 **PREDICT BEST CROP**

RECOMMENDED CROP

 **Maize**

Match Score: **7.1%**

ALL CROPS RANKED BY MATCH SCORE

#1	 Maize		7.1%
#2	 Jute		6.7%
#3	 Rice		6.5%
#4	 Pigeonpeas		6.0%
#5	 Papaya		5.9%
#6	 Cotton		5.7%
#7	 Lentil		5.1%
#8	 Chickpea		5.0%
#9	 Mothbeans		4.9%
#10	 Watermelon		4.8%

OPTIMAL CONDITIONS FOR MAIZE

NITROGEN ●

**77.8
mg/kg**

Range: 60-100 mg/kg
Your input: 80

PHOSPHORUS ●

**48.4
mg/kg**

Range: 35-60 mg/kg
Your input: 62

POTASSIUM ●

**19.8
mg/kg**

Range: 15-25 mg/kg
Your input: 96

TEMPERATURE ●

22.4 °C

Range: 18-26.5 °C
Your input: 25

HUMIDITY ●

65.1 %

Range: 55.3-74.8 %
Your input: 28

10. Conclusion

This project demonstrates the effective use of data mining and machine learning techniques in solving a real-world agricultural problem. The main objective was to help farmers select the most suitable crop based on soil nutrients and environmental conditions. Various features such as Nitrogen, Phosphorus, Potassium, temperature, humidity, pH, and rainfall were analyzed. These factors play a crucial role in determining crop growth and productivity. The dataset was properly preprocessed by handling duplicates, encoding labels, and scaling features. This ensured better data quality and improved model performance.

Three machine learning algorithms were implemented: Decision Tree, Random Forest, and K-Nearest Neighbors. Each algorithm was trained and tested using the dataset. Among them, Random Forest achieved the highest accuracy and gave the best results. This is because it combines multiple decision trees and reduces overfitting.

Data warehouse concepts such as Star Schema and Snowflake Schema were also designed. These schemas helped in organizing the data efficiently for analysis. Visualization techniques like pie charts, bar graphs, and heatmaps were used. They provided better understanding of patterns and relationships in the data.

The results show that soil nutrients and climate conditions strongly influence crop selection. The model can successfully predict the most suitable crop with good accuracy. This system can help farmers make better decisions and reduce financial losses. It also saves time and improves agricultural productivity.

Overall, this project highlights the importance of data-driven decision making in agriculture. It proves that machine learning can be effectively used to solve practical problems.

Future Scope:

- Use real-time soil sensor data for dynamic predictions
- Integrate weather API for live climate data
- Deploy as a mobile app for farmers
- Add yield quantity prediction (regression) along with crop type

11. References

1. Kaggle Crop Recommendation Dataset:

<https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset>

2. Working Model Reference - Claude

<https://claude.ai/login>